

Lojistik Regresyonun Matematiksel Arka Planı

Kodlamadan önce sindirilecek temel çalışma kağıdı — Miuul Makine Öğrenmesi Modülü

Bu çalışma kağıdının amacı: Lojistik regresyonda kullanılan sigmoid fonksiyonu, log loss (cross entropy) maliyet fonksiyonu ve bunun gradient descent ile nasıl minimize edildiğini, koduna geçmeden önce elle hesaplayarak kalıcı biçimde oturtmak. Doğrusal regresyondaki gradient descent notlarının doğal devamıdır — oradaki temel mantık (eğim bul, ters yönde adım at) burada da aynen geçerli, değişen tek şey hangi fonksiyonun eğimini aldığımız.

1 Problem Nedir?

Doğrusal regresyonda hedef sürekli bir sayı tahmin etmektir (satış = 14.3 gibi). Lojistik regresyonda hedef bir **sınıf** tahmin etmek: hasta mı değil mi, churn oldu mu olmadı mı gibi 0/1 etiketler. Doğrudan $b + w \cdot x$ formülünü kullanamayız çünkü bu ifade $-\infty$ ile $+\infty$ arası her değeri üretebilir; oysa biz "1 sınıfına ait olma olasılığı" istiyoruz, bunun 0 ile 1 arasında olması gerekir.

Sigmoid fonksiyonu

SİGMOİD (LOJİSTİK FONKSİYON)

$$\hat{y} = 1 / (1 + e^{-z}) \quad \text{burada} \quad z = b + w_1x_1 + w_2x_2 + \dots + w_px_p$$

Önce klasik doğrusal skor z 'yi hesaplarız (buna "logit" da denir), sonra bu skoru sigmoid fonksiyonundan geçiririz. Sigmoid'in işi basit: içine ne verirsene ver, çıktıyı her zaman 0 ile 1 arasına sıkıştırır. Grafiği S harfi şeklindedir; $z=0$ civarında hızlı geçiş yapar, uçlarda 0'a veya 1'e yapışır.

SEZGİ / BENZETME

z , "ham skor" gibi düşünülebilir — ne kadar büyükse o kadar "1 sınıfına yakın", ne kadar küçük (negatif) ise o kadar "0 sınıfına yakın". Ama z 'nin kendisi bir olasılık değil, sınırsız bir sayı. Sigmoid bu sınırsız sayıyı alıp "olasılık dili"ne çevirir: z çok büyükse $\hat{y} \rightarrow 1$, z çok küçükse (çok negatif) $\hat{y} \rightarrow 0$, $z=0$ ise $\hat{y}=0.5$ (tam kararsız nokta).

Sayısal örnek — elle hesaplama

$b=5, w_1=4, w_2=-4, w_3=3 \mid x_1=2, x_2=3, x_3=0$

1 $z = 5 + (4 \cdot 2) + (-4 \cdot 3) + (3 \cdot 0) = 5 + 8 - 12 + 0 = 1$

2 $\hat{y} = 1 / (1 + e^{-1}) = 1 / (1 + 0.368) = 0.731$

Yorum: bu gözlem biriminin "1 sınıfına ait olma olasılığı" %73.1. Eşik 0.5 kabul edilirse, model bu gözlemi **1 sınıfına** atar.

ALİŞTIRMA 1

Aynı b, w değerleriyle $x_1=0$, $x_2=0$, $x_3=5$ için $\hat{y}$ 'yi hesapla.

CEVAP ANAHTARI

$z = 5 + (4 \cdot 0) + (-4 \cdot 0) + (3 \cdot 5) = 5 + 15 = 20$. $\hat{y} = 1/(1+e^{-20}) \approx 1/(1+0.0000000021) \approx \mathbf{0.9999999979}$, yani neredeyse kesin 1 sınıfı. z çok büyüdüğünde sigmoid'in 1'e "yapıştığını" burada görüyorsun.

2 Maliyet Fonksiyonu Neden MSE Değil?

Doğrusal regresyonda b, w'yi bulmak için hatayı (tahmin-gerçek)² olarak ölçüp (MSE) minimize ediyorduk. Bu, düz bir doğru ($h(x)=b+wx$) için **tek çukurlu (konveks)** bir yüzey oluşturuyordu — gradient descent güvenle dibe inebiliyordu.

NEDEN?

Sigmoid, S-şekilli ve kendi içinde bükülen (eğrisel) bir fonksiyon. Bunu MSE ile birleştirirsen — yani $(\hat{y}-y)^2$ alırsan, burada $\hat{y} = \text{sigmoid}(z)$ — ortaya çıkan hata yüzeyi artık tek çukurlu olmuyor, **tümsekli (non-konveks)** hale geliyor. Gradient descent böyle bir yüzeyde bir çukura düşüp "buldum minimum" diyebilir, ama bu sadece **yerel minimum** olabilir, gerçek en iyi nokta değil. Bu yüzden lojistik regresyonda MSE yerine **log loss** kullanılır — log loss + sigmoid kombinasyonu matematiksel olarak yine tek çukurludur (bu çalışma kağıdının 4. bölümünde bu neden-sonuç ilişkisini elle göreceksin).

3 Log Loss (Cross Entropy) Formülü

TEK GÖZLEM İÇİN LOG LOSS

$$\text{Loss}_i = -y_i \cdot \log(\hat{y}_i) - (1-y_i) \cdot \log(1-\hat{y}_i)$$

TÜM VERİ SETİ İÇİN ORTALAMA (M GÖZLEM)

$$\text{Log Loss} = (1/m) \cdot \sum_{i=1}^m [-y_i \cdot \log(\hat{y}_i) - (1-y_i) \cdot \log(1-\hat{y}_i)]$$

Parça parça okuma — formül neden "anahtarlanır" gibi çalışır

Bu formülün püf noktası: her gözlemde gerçek değer y_i ya **0** ya **1** olduğu için, formülün iki yarısından her zaman biri "kapanır" (sıfırlanır), diğeri devrede kalır.

Durum A: $y_i = 1$ ise

- 1 Formülün sol tarafı: $-1 \cdot \log(\hat{y}) \rightarrow$ devrede kalır.
- 2 Formülün sağ tarafı: $(1-1) \cdot \log(1-\hat{y}) = 0 \cdot (...) = \mathbf{0} \rightarrow$ tamamen gider.

- 3 Geriye kalan: $\text{Loss} = -\log(\hat{y})$. Yani sadece "1 sınıfına ait olma olasılığı $\hat{y}$ ne kadar yüksekse o kadar iyi" değerlendirmesi kalıyor.

Durum B: $y_i = 0$ ise

- 1 Formülün sol tarafı: $-0 \cdot \log(\hat{y}) = 0 \rightarrow$ tamamen gider.
- 2 Formülün sağ tarafı: $-(1-0) \cdot \log(1-\hat{y}) = -\log(1-\hat{y}) \rightarrow$ devrede kalır.
- 3 Not: $(1-\hat{y})$, "0 sınıfına ait olma olasılığı" demektir. Yani burada da aynı mantık işliyor, sadece 0 sınıfı için.

SEZGİ / BENZETME

Formülü bir "anahtarlama devresi" gibi düşün: y_i hangi sınıfı işaret ediyorsa, formül otomatik olarak o sınıfın olasılığını değerlendiren yarıyı açık bırakıp diğerini kapatıyor. Elle hesap yaparken "hangi tarafın hayatta kaldığını" bulmak, hesabın yarısını anında bitirir.

Neden eksi log? — tek bir sayı ile sezgi

Durum	$\log(\text{değer})$	$-\log(\text{değer}) = \text{Loss}$	Yorum
$\hat{y} = 0.99$ (neredeyse emin, doğru)	$\log(0.99) \approx -0.01$	≈ 0.01 (çok küçük)	Model doğru ve emin $\rightarrow$ neredeyse hiç ceza yok
$\hat{y} = 0.50$ (kararsız)	$\log(0.50) \approx -0.69$	≈ 0.69 (orta)	Model kararsız $\rightarrow$ orta ceza
$\hat{y} = 0.01$ (neredeyse emin, ama yanlış)	$\log(0.01) \approx -4.6$	≈ 4.6 (çok büyük)	Model emin ama yanlış $\rightarrow$ ağır ceza

NEDEN?

$\log$ fonksiyonu 0'a yaklaştıkça $-\infty$ 'a gider. Bu yüzden model "yanlış ama çok emin" davrandığında (gerçek sınıfa neredeyse sıfır olasılık verdiğinde) ceza patlarcasına büyür. MSE'de böyle keskin bir ceza mekanizması yok — büyük hatalar sadece "kareleniyor", $\log$ loss'ta ise üstel biçimde cezalandırılıyor. Bu, modelin aşırı özgüvenli ve yanlış tahminlerden kaçınmasını sağlar.

Sayısal örnek

3 gözlemlik mini veri seti:

Gözlem	y (gerçek)	$\hat{y}$ (tahmin edilen olasılık)	Hangi yarı devrede?	Loss_i
1	1	0.80	sol ($-\log(\hat{y})$)	$-\log(0.80) \approx 0.223$
2	0	0.30	sağ ($-\log(1-\hat{y})$)	$-\log(0.70) \approx 0.357$
3	1	0.95	sol ($-\log(\hat{y})$)	$-\log(0.95) \approx 0.051$

Ortalama Log Loss = $(0.223 + 0.357 + 0.051) / 3 = 0.631 / 3 \approx \mathbf{0.210}$

ALİŞTİRMA 2

Aynı formatta, $y=0$, $\hat{y}=0.10$ olan bir gözlem için $Loss_i$ 'yi hesapla. Hangi yarı devrede kalıyor?

CEVAP ANAHTARI

$y=0$ olduğu için sağ taraf devrede: $Loss = -\log(1-0.10) = -\log(0.90) \approx \mathbf{0.105}$. Model gerçek sınıfa (0) yüksek olasılık (0.90) verdiği için ceza küçük — beklendiği gibi.

4 Neden Log Loss + Sigmoid Tek Çukurlu Kalıyor?

Bu bölüm, 2. bölümdeki "neden MSE değil" sorusunun matematiksel temelini gösteriyor. İki fonksiyonu karşılaştıralım.

Log loss + sigmoid: log, üsteli "yiyor"

Sigmoid: $\hat{y} = 1/(1+e^{-z})$. Bunu log loss'un içine koyunca:

$$\log(\hat{y}) = \log(1/(1+e^{-z})) = -\log(1+e^{-z})$$

log fonksiyonu, sigmoid'in içindeki e^{-z} 'yi düzleştirir — üstel terim log ile birleşince yumuşak, pürüzsüz bir ifadeye dönüşür (bu ifadeye "softplus" denir, tek çukurludur). Bunu $z = b+wx$ (b, w'ye göre doğrusal) ile birleştirdiğinde, ortaya çıkan toplam ifade b ve w'ye göre de **tek çukurlu (konveks)** kalır.

MSE + sigmoid: kare alınca eğrisellik katlanıyor

$$\text{Hata} = (\hat{y} - y)^2 = (1/(1+e^{-z}) - y)^2$$

Burada log yok — sigmoid'in S-eğrisi olduğu gibi kareye alınıyor. Sigmoid zaten kendi içinde bükülen (dönüş noktası olan) bir eğri; bunu kareye alınca eğrisellik yok olmuyor, katlanıyor. Sonuçta ortaya çıkan yüzey tek çukur değil, birden fazla iniş-çıkışı olan tümsekli bir yüzey oluyor.

NEDEN?

Kısacası: **log** ve **e^{-z}** doğrudan birbirini iptal ediyor ($\log(e^x)=x$ mantığı), geriye düz, tek çukurlu bir ifade kalıyor. MSE'de ise hiçbir iptal olmuyor, sigmoid'in eğrisi kareyle birlikte katlanarak tümsekli hale geliyor. Bu yüzden lojistik regresyonda "yerel minimuma düşme riskine karşı ekstra kontrol" gerekmiyor — problem, doğru hata fonksiyonu (log loss) seçilerek baştan ortadan kaldırılıyor.

5 Gradient Descent ile b, w'yi Bulmak

Doğrusal regresyondaki gradient descent notlarını hatırla: $J(\theta)$ fonksiyonunun ilgili parametreye göre kısmi türevini alıp, bu türevi öğrenme oranıyla çarparak parametreyi güncelliyorduk. Lojistik regresyonda da mantık birebir aynı — sadece $J(\theta)$ artık log loss.

GENEL GÜNCELLEME KURALI (REPEAT)

$$\theta_j := \theta_j - \alpha \cdot \partial J(\theta) / \partial \theta_j$$

Önemli farkındalık: b ve w değerleri "gökten gelmiyor" — bir yöntemle bulunması gerekiyor. Bunu gradient descent (iteratif, adım adım) ile bulabileceğimiz gibi, istatistikte **maksimum olabilirlik**

(**maximum likelihood**) yöntemiyle de bulabiliriz. İkisi de aynı hedefe (log loss'u minimize eden b, w) farklı yollarla ulaşır. Biz burada gradient descent'e odaklanıyoruz çünkü kodda bunu sıfırdan yazacaksın.

Akış — doğrusal regresyondan farkı sadece 1 satır

Adım	Doğrusal Regresyon	Lojistik Regresyon
Tahmin	$\hat{y} = b + w \cdot x$	$\hat{y} = \text{sigmoid}(b + w \cdot x)$
Maliyet fonksiyonu	MSE	Log Loss
Kısmi türev alma	MSE'nin türevi	Log Loss'un türevi
Güncelleme kuralı	$\theta := \theta - \alpha \cdot \text{eğim}$	$\theta := \theta - \alpha \cdot \text{eğim}$ (birebir aynı)
İterasyon (tekrar)	for döngüsü	for döngüsü (birebir aynı)

SEZGİ / BENZETME

Hocanın vurguladığı nokta tam olarak bu: doğrusal regresyonda gradyanı çok detaylı hesapladık, "ağırlıklar değiştiğinde fonksiyonun değerini minimum yapacak yöne gitme" mantığını oturttuk. Lojistik regresyonda bu mekanizmanın kendisi **değişmiyor** — sadece hangi fonksiyonun ($J(\theta)$) eğimini aldığımız değişiyor. Motoru değiştirmiyoruz, sadece pusulanın hangi haritaya baktığını değiştiriyoruz.

Kısmi türevin sonucu (ezbersiz, sonucu bil yeter)

Log loss'un z 'ye göre türevi alınıp zincir kuralıyla b ve w 'ye yayıldığında, şaşırtıcı derecede sade bir sonuç çıkar:

$$\partial J / \partial b = (1/m) \cdot \sum_{i=1}^m (\hat{y}_i - y_i) \quad \partial J / \partial w = (1/m) \cdot \sum_{i=1}^m (\hat{y}_i - y_i) \cdot x_i$$

NEDEN?

Bu formül doğrusal regresyondaki $(h(x)-y)$ ve $(h(x)-y) \cdot x$ formülleriyle **görsel olarak birebir aynı** görünüyor — tek fark $\hat{y}$ artık sigmoid'den geçmiş bir değer. Bu tesadüf değil: log loss özellikle bu sade türevi verecek şekilde tasarlanmıştır (log ve exp'in birbirini iptal etmesinin doğrudan sonucu). Pratik anlamı: kodda yazdığın `update_weights` fonksiyonunun iskeleti (`b_deriv_sum`, `w_deriv_sum`, `learning_rate` ile güncelleme) neredeyse hiç değişmeyecek; tek değişen, $\hat{y}$ 'yi hesaplarken artık $b+w \cdot x$ 'i sigmoid'den geçirmen.

ALİŞTIRMA 3

Elinde 2 gözlemlik veri var: $(x=1, y=1, \hat{y}=0.7)$ ve $(x=2, y=0, \hat{y}=0.4)$. $\partial J / \partial b$ ve $\partial J / \partial w$ 'yi elle hesapla ($m=2$).

CEVAP ANAHTARI

$(\hat{y}-y)$ değerleri: gözlem 1 $\rightarrow 0.7-1 = -0.3$, gözlem 2 $\rightarrow 0.4-0 = 0.4$.

$\partial J/\partial b = (-0.3 + 0.4)/2 = 0.1/2 = \mathbf{0.05}$

$(\hat{y}-y) \cdot x$ değerleri: gözlem 1 $\rightarrow -0.3 \cdot 1 = -0.3$, gözlem 2 $\rightarrow 0.4 \cdot 2 = 0.8$

$\partial J/\partial w = (-0.3 + 0.8)/2 = 0.5/2 = \mathbf{0.25}$

Bu iki sayı, öğrenme oranıyla çarpılıp b ve w'den çıkarılacak — tıpkı doğrusal regresyonda yaptığın gibi.

6 Bu Matematiğin Koda Bağlanması

Matematikteki adım	Kodda karşılığı
$z = b + w \cdot x$ hesaplama	daha önce yazdığın satırla birebir aynı
$\hat{y} = \text{sigmoid}(z)$	yeni satır: <code>1 / (1 + np.exp(-z))</code>
Log Loss hesaplama	<code>cost_function</code> içinde $-y \cdot \log(\hat{y}) - (1-y) \cdot \log(1-\hat{y})$ toplamının ortalaması
$\partial J/\partial b, \partial J/\partial w$	<code>update_weights</code> içindeki <code>b_deriv_sum</code> , <code>w_deriv_sum</code> — formül aynı, sadece $\hat{y}$ artık sigmoid'den geçmiş
$\theta := \theta - \alpha \cdot \text{eğim}$	<code>b = b - learning_rate * eğim_b</code> ; <code>w = w - learning_rate * eğim_w</code> (değişmedi)
Tekrar (iterasyon)	<code>for</code> döngüsü (değişmedi)
0.5 eşiğiyle sınıf kararı	yeni satır: $\hat{y} \geq 0.5$ ise sınıf 1, değilse sınıf 0

Önemli hatırlatma: `LogisticRegression().fit()` sklearn'de bu iteratif süreci farklı optimizasyon algoritmalarıyla (örn. lbfgs) çok hızlı şekilde çözer; ayrıca maksimum olasılık temelli çözümler de kullanılabilir. Senin sıfırdan yazacağın gradient descent, aynı log loss'u minimize eden b, w'ye adım adım, daha yavaş ama **görerek** ulaşacak — amaç hız değil, mekanizmayı kavramak.

7 Kodlamaya Geçmeden Önce Kontrol Listesi

- ☐ Sigmoid fonksiyonunun neden gerekli olduğunu (z'yi 0-1 aralığına sıkıştırmak) açıklayabiliyorum.
- ☐ $z = b + w \cdot x$ ile $\hat{y} = \text{sigmoid}(z)$ arasındaki farkı biliyorum: biri ham skor, diğeri olasılık.
- ☐ Log loss formülünde $y=1$ ve $y=0$ durumlarında hangi yarının "devrede kaldığını" elle gösterebiliyorum.
- ☐ $-\log$ fonksiyonunun neden "emin ama yanlış" tahminleri ağır cezalandırdığını sayısal örnekle açıklayabiliyorum.
- ☐ MSE+sigmoid'in neden tümsekli (non-konveks), log loss+sigmoid'in neden tek çukurlu (konveks) olduğunu, log/exp ilişkisi üzerinden anlatabiliyorum.
- ☐ Log loss'un b ve w'ye göre türevinin, doğrusal regresyondaki türevle neden bu kadar benzediğini biliyorum.

- ☐ Gradient descent'in genel akışının (eğim bul, öğrenme oranıyla çarp, çıkar, tekrarla) doğrusal regresyondan hiç değişmediğini, sadece hangi fonksiyonun türevinin alındığının değiştiğini anlıyorum.
- ☐ Elle en az 1 sigmoid ve 1 log loss hesabını, hatasız şekilde yaptım.

Hepsini işaretleyebiliyorsan, kodlama kısmına geçmeye hazırsın. Takıldığın alıştırma olursa çözümünü getir, birlikte kontrol edelim.